

Track 2 Model Testing Plan — Dev100 → Dev10k Funnel

Current Results

Completed: Sherpa Zipformer Standard (~70M) on Dev10k

Metric	Value	Notes
CER	29.94%	Better than official baseline (34.59%)
WER	47.08%	Better than official baseline (52.77%)
TTFT P50	???	Latency eval broken — wrong manifest CSV
TTLT P50	???	Same issue — needs <code>Dev_streaming.csv</code>
RTF	< 1.0×	Proven CPU-viable

Reference Baselines (from Bernard's Track 1 GPU eval, full Dev 47,929 utts)

Model	CER	WER	Notes
Qwen3-ASR 1.7B	14.23%	23.34%	Best accuracy, too slow for CPU
Whisper large-v2	20.36%	26.08%	Non-streaming, GPU only
Whisper large-v3	23.01%	32.23%	Worse than v2 on dysarthric

Official Track 2 Baseline (Test1, CPU)

Model	CER	WER	TTFT P50	TTLT P50
Zipformer (Librispeech, zero-shot)	34.59%	52.77%	1025 ms	423 ms

Our Sherpa standard already beats the official baseline on accuracy (29.94 vs 34.59 CER) without any fine-tuning. This is a strong starting point.

Blocking Issue: Fix Latency Evaluation First

Before testing more models, fix the latency pipeline so every model gets full metrics:

1. Download `Dev_streaming.csv` from Box — the organizers' pre-computed streaming manifest with `mfa_speech_start` column

2. **Re-run latency eval for Sherpa standard** using the correct manifest to get baseline TTFT/TTLT numbers
3. **Fix the TTLT negativity bug** — check if all TTLT values are < 0 and getting filtered. If so, either remove the `>= 0` filter for local dev or investigate the timing in `local_decode.py`

This must be done before testing other models, otherwise you're comparing accuracy-only numbers and can't make Pareto decisions.

Testing Strategy: Dev100 → Dev10k Funnel

Yes, this approach makes sense. Dev100 (~100 utterances) gives you a quick 5-10 minute smoke test per model to check:

- Does it produce non-empty output on dysarthric speech?
- Is RTF < 1.0 on CPU?
- Is CER roughly in the right ballpark?

Models that pass Dev100 get promoted to Dev10k (~10,521 utterances) for official scoring. This saves hours of compute on models that are obviously non-viable.

Dev100 Gate Criteria (must pass ALL to proceed to Dev10k)

- RTF $< 1.5\times$ on CPU (some margin — Dev10k will give more accurate RTF)
- Non-empty output rate $> 90\%$
- CER $< 60\%$ (rough sanity check — models above this are useless)
- Model loads and runs without crashes

Dev10k Gate Criteria (must pass to be a submission candidate)

- CER $< 35\%$ (must beat or match official baseline)
 - Latency evaluated with correct streaming manifest
 - Total runtime extrapolates to $< 15,000$ s for test set
-

Models to Test (Priority Order)

1. Sherpa Zipformer — Kroko variant ★ HIGH PRIORITY

Status: Not yet tested **Why next:** Same wrapper as standard Zipformer (just change `config.yaml`), edge-optimised Zipformer2 architecture. May give better accuracy at similar or lower latency. Zero additional integration work.

Dev100 steps:

```

bash

# 1. Switch variant in config
cd track2_starting_kit/sherpa_zipformer
sed -i 's/variant: "standard"/variant: "kroko"/' config.yaml

# 2. Download weights
bash setup.sh

# 3. Run local_decode on Dev100
python3 ../local_decode.py \
  --submission-dir . \
  --manifest-csv /workspace/SAPC2/manifest/Dev100.csv \
  --data-root /workspace/SAPC2 \
  --out-csv ./Dev100.kroko.predict.csv \
  --out-partial-json ./Dev100.kroko.partial_results.json

# 4. Score
bash /workspace/SAPC2/evaluate.sh \
  --start_stage 2 --stop_stage 2 \
  --split Dev100 \
  --hyp-csv ./Dev100.kroko.predict.csv

```

If passes Dev100 → run Dev10k with same commands, swap Dev100→Dev10k

2. Sherpa Zipformer — Small variant (~20M) ★ PARETO ANCHOR

Status: Not yet tested **Why:** 3.5× smaller than standard. If accuracy is acceptable (even after fine-tuning), the latency advantage could dominate the low-latency end of the Pareto frontier. This tests the "fast but less accurate" strategy.

Dev100 steps:

```

bash

# Same as Kroko but with variant: "small"
sed -i 's/variant: ".*"/variant: "small"/' config.yaml
bash setup.sh

# Then same local_decode + evaluate commands

```

Expected: Worse zero-shot CER than standard, but much lower RTF. The key question is *how much worse* — if CER > 50%, it's only viable with fine-tuning.

3. Moonshine tiny (~30M) and base (~100M)

Status: Not yet tested **Why:** Purpose-built for CPU/edge. Different architecture family from Zipformer. Could provide diversity on the Pareto frontier if accuracy is reasonable.

Dev100 steps:

```
bash

# 1. Ensure wrapper exists in track2_starting_kit/moonshine/
# 2. Install
pip install useful-moonshine-onnx

# 3. Run local_decode
python3 local_decode.py \
  --submission-dir ./moonshine \
  --manifest-csv /workspace/SAPC2/manifest/Dev100.csv \
  --data-root /workspace/SAPC2 \
  --out-csv ./Dev100.moonshine_tiny.predict.csv \
  --out-partial-json ./Dev100.moonshine_tiny.partial_results.json

# 4. Score
```

Risk: Fine-tuning path is undocumented for v2 (Bernard's note). If zero-shot CER is bad (> 45%), deprioritise unless a fine-tuning path emerges.

Test both tiny and base — tiny may be viable as a latency anchor even with worse accuracy.

4. GigaAM 100M (Sber)

Status: Not yet tested. Setup script exists in `track2_starting_kit/gigaam/`. **Why:** SSL-pretrained, chunkwise attention for streaming, MIT license. Bernard recommended it.

Dev100 steps:

```
bash
```

```
# 1. Run setup
cd track2_starting_kit/gigaam
bash setup.sh

# 2. CRITICAL: First verify English works
python3 test_streaming.py --audio /workspace/SAPC2/processed/Dev/<some_english_utt>.
# If output is Russian/garbage → SKIP this model immediately

# 3. If English works, run local_decode on Dev100
python3 ../local_decode.py \
  --submission-dir ./gigaam \
  --manifest-csv /workspace/SAPC2/manifest/Dev100.csv \
  --data-root /workspace/SAPC2 \
  --out-csv ./Dev100.gigaam.predict.csv \
  --out-partial-json ./Dev100.gigaam.partial_results.json
```

Gate: If English output is garbage or empty, skip immediately — don't waste time on Dev10k.

5. Whisper small/base (faster-whisper INT8) — DEPRIORITISED

Status: Not yet tested **Why deprioritised:** Encoder-decoder architecture gives terrible TTFT in streaming. Bernard explicitly warned that Pareto scoring (which averages TTFT and TTLT) will penalise this heavily. Only test if models 1-4 all fail to reach acceptable accuracy.

Only test if: All Zipformer variants have CER > 35% and Moonshine/GigaAM are non-viable.

6. Voxtral Realtime (4B) — SMOKE TEST ONLY

Status: Not yet tested **Why:** Bernard initially suggested it. 4B params on CPU is almost certainly too slow.

Action: Single-utterance RTF test on CPU. If RTF > 10×, discard and document. Do NOT run Dev100.

Execution Order

```
Step 0: Fix latency eval (get Dev_streaming.csv, re-score Sherpa standard)
Step 1: Kroko Zipformer → Dev100 → if pass → Dev10k
Step 2: Small Zipformer → Dev100 → if pass → Dev10k
Step 3: Moonshine tiny → Dev100 → if pass → Dev10k
Step 4: Moonshine base → Dev100 → if pass → Dev10k
Step 5: GigaAM English check → if English OK → Dev100 → if pass → Dev10k
```

Step 6: Voxtral single-utterance smoke test (discard if RTF > 10×)

Step 7: (Only if needed) Whisper small → Dev100

Estimated time: Steps 1-2 take ~15 min each on Dev100, ~2-3 hours each on Dev10k. Steps 3-5 add setup time. Total: ~1 day for full funnel.

Results Tracking Table

Fill in as each model is tested:

Model	Params	Dev100 CER	Dev100 WER	RTF	Dev10k CER	Dev10k WER	TTFT P50	TTLT P50	Status
Sherpa standard	~70M	—	—	< 1.0	29.94%	47.08%	???	???	✅ Accuracy done, latency blocked
Sherpa kroko	TBD								❌ Not started
Sherpa small	~20M								❌ Not started
Moonshine tiny	~30M								❌ Not started
Moonshine base	~100M								❌ Not started
GigaAM	~100M								❌ Not started
Voxtral	4B	—	—		—	—	—	—	❌ Smoke test only

After Testing: Next Steps

1. **Pick top 2-3 models** based on Pareto analysis (CER vs latency)
2. **Fine-tune the best Zipformer variant** on SAP training data (742h) via Icefall/k2 on GPU — this is the critical path to winning

3. **Submit to Codabench** and verify runtime < 15,000s on test1

4. **Iterate** based on test1 results

The fine-tuning step is where the real accuracy gains come from. The SAPC1 winner went from 36.3% → 23.7% WER with fine-tuning. Our Zipformer at 29.94% CER zero-shot should improve dramatically with 742h of dysarthric training data.